DeepL
订阅DeepL Pro以翻译大型文件。
欲了解更多信息，请访问www.DeepL.com/pro。

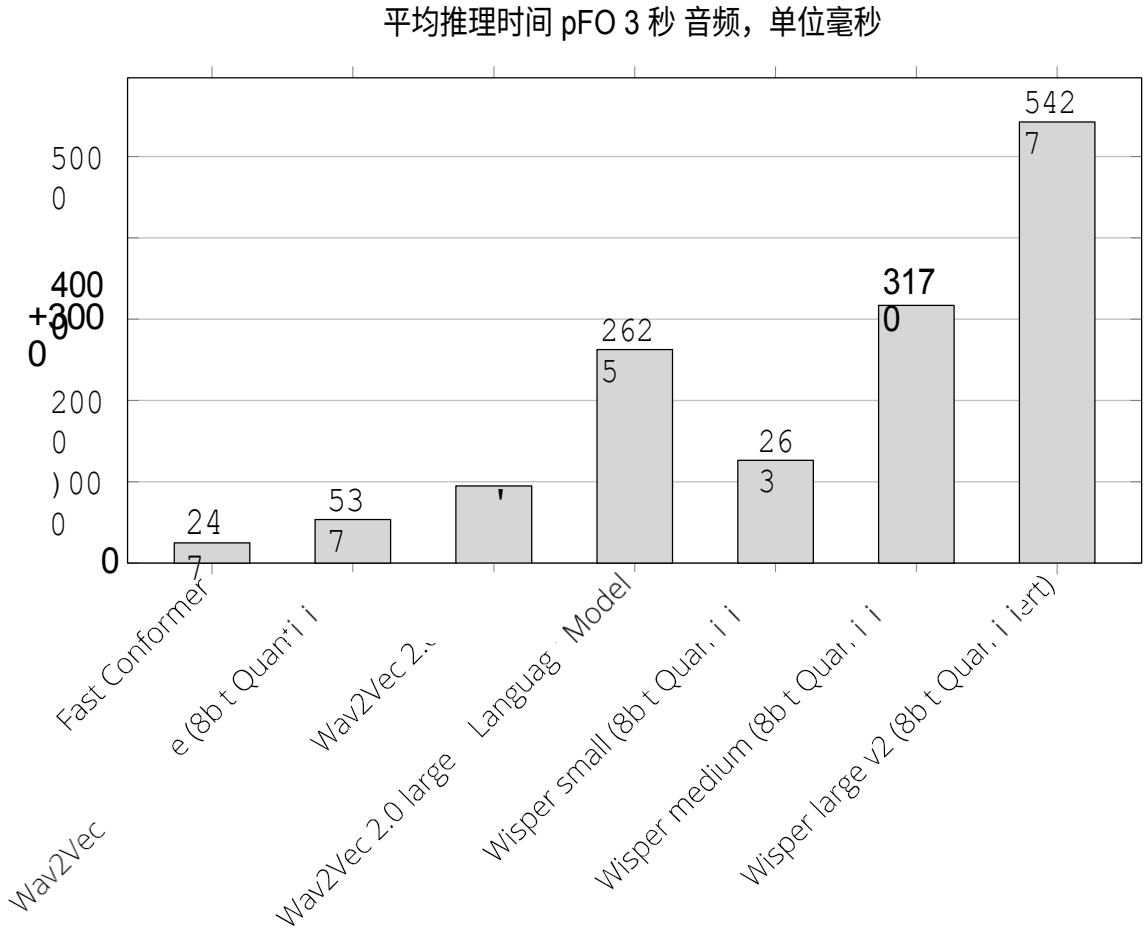


图 5.3：模型不同优化步骤的推理时间比较。测试在英特尔 i5-J 240p 上进行。所示数值为 50 次迭代的平均值。

实施了各种性能优化措施。其中一些措施应用于 2022 年发布的 Whisper 型号。最后，我们将这些模型的推断速度与 2023 年推出的 Fast Conformer 进行了比较。

研究了提高推理速度的三种方法：训练较小的模型、去除后处理步骤以及通过量化优化硬件。这些策略的在给定的硬件限制条件下，最大限度地提高模型的效率和性能

Wav2Vec 2.0

为了优化德语语音识别我们对 Wav2Vec 2 模型的两个变体进行了不同规模的训练。基础 "模型有 9440 万个参数[7]，"大型 "模型有 3.15 亿个参数[8]。这些模型代表

7https://huggingface.co/oliverguhr/wav2vec2 -base-german-cv9
8https://huggingface.co/oliverguhr/wav2vec2-large-xlsr-53-german-cv9

是德国 Wav2Vec 2.0 1B 型号的更高效替代品，拥有 10 亿
参数
对这些模型转录准确性的比较表明，域内数据的词错误率（WER）仅相差 2.5 个百分点（见表 5.2）。然而，域外数据的差异更为明显，IB 模型的词错误率为 15%，Large 模型为 19%，Base 模型为 27%。这些结果表明，模型的概括能力随着模型规模的减小而降低。
总体而言，Wav2Vec Large 模型在域内数据方面的 WER 值比 1B 模型高 1 个百分点，而在域外数据方面则落后于 1B 模型 4 个百分点。同时，它的参数只有 1B 模型的三分之一。这使它成为参数数量和转录精度之间的有效折衷。基于这些特性，Wav2Vec Large 模型被选为进一步优化的基础。
在默认配置中，Wav2Vec 2.0 模型使用基于 KenLM [Heall] 的语言模型来重建最可能的单词序列，并尽量减少转录错误。然而，正如第 6.2 章所述，用于语言处理的模型（处理转录）对转录错误具有鲁棒性。因此，KenLM 语言模型可以省略，因为语义搜索可以在一定程度上弥补错误。
图 5.3 显示，这种方法将推理速度提高了 2.8 倍，推理时间从 2 625 毫秒缩短到 948 毫秒。
对 Wave2Vec Large 模型进行的第三项重要优化是量化。在神经网络中，量化是降低网络组件权重和激活的精度过程。这涉及降低模型执行计算的分辨率。在 Wav2Vec 2.0 Large 模型的具体案例中，这意味着模型的前馈层使用无符号 8 位整数，而不是 32 位浮点数。这一修改既减少了内存需求，也降低了网络的计算需求。
这一优化使推理时间从 948 毫秒到 537 毫秒。同时，模型的内存消耗也从 1 300 MB 减少到 317 MB。这一量化的 Wave2Vec 2.0 模型 2023 年之前一直用于开发和测试 VUI。第 8 章所述的虚拟用户界面评估也是通过该模型进行的。

低语

图 5.3 比较了经过优化的 Wav2Vec 2.0 模型与后来发布的 Whisper 和 Fast Conformer 模型推理速度。
这次比较使用了不同规模的 Whisper 模型：小型（2.44 亿个参数）、中型（7.69 亿个参数）和大型-v2（1.55 亿个参数）。这些模型也进行了量化。优化后的 Wav2Vec
2.0 模型比测试的最小 Whisper Small 模型快 .4 倍，后者推理时间为 1,263

毫秒。此外，Wav2Vec 2.0 模型在域内和域外都取得了更好的结果。

数据集。只有较大的 Whisper Medium 和 Whisper Large v2 模型比 Wav2Vec 2.0 模型取得了更好的结果，但推理速度分别慢了 6 倍和 10 倍。由于 Whisper 模型对噪声具有很强的鲁棒性，因此在 VUI 中使用 Whisper 模型很有意义[Rad+22d.但是，如果没有进一步优化或硬件加速推理，这些模型就不太适合 6iÜZUStUf6Ü 这一特定应用。

快速成型器

Fast Conformed [Rek+23] 是 Conformed 架构 [Gul+20] 在效率方面的优化版本。根据作者的说法，该模型的速度原始符合架构的 2.8 倍。

与经过优化的 Wav2Vec 2.0 模型相比，Fast Conformer 模型速度快 2.2 倍，推断一个 3000 毫秒的音频序列只需 247 毫秒。此外，如表 5.2 和表 5.3 所示，Fast Conformer 模型域内数据集 WER 值低 1%，域外数据集 WER 值低 2%。

这些结果表明，快速构形器的推理速度明显更快，转录也更精确。由于执行速度快、识别率，因此 VUI 采用了快速 Conformer 架构的德国模型。

6 语言处理

6.1 从文本到意图

语言处理的主要任务是从 ASR 转录的文本中提取知识。在科学文献中，这项任务通常被自然语言处理（NLP）或自然语言理解（NLU）。提取这些知识的方法源于计算语言学和机器学习。与虚拟用户界面特别相关子任务尤其值得注意。这些子任务包括神经搜索、命名实体识别、文本分割和分类。

在虚拟用户界面中，把握用户语句含义的能力起着决定性作用。例如，如果我们考虑到用户向机器人发出的请求，如 "移动到客厅"，则必须提取两个基本信息才能做出适当的响应：具体动作 "移动机器人 "和目标 "客厅"。这一识别预期行动的过程在专业文献中被称为意图识别或意图分类。识别特定实体（如文本中的位置、名称或时间规格）被称为命名实体识别（NER）。正如第 3.J 章所述，虚拟用户界面的可用性与其正确解释释义的能力直接相关。该领域最大的挑战之一是如何映射人类语言固有的多变性。必须确保即使指令的表述与系统中存储的表述不同，系统也能做出正确的反应。只识别 "去 X "等固定模式的方法在面对 "去客厅 "等多变的指令时会失效。这些系统迫使用户学习特定的指令，这在很多方面都问题。随着虚拟用户界面的复杂程度增加，用户的学习难度也会成正比增加。此外，不熟悉特定命令的用户在与 SAR 交互时也会受到很大限制。本论文中用于应对这些挑战的技术将在第 6.2 节中详细介绍。

另一项任务是对语句进行分段。通常情况下，ASR 提供

不带标点符号的语句誊本。如果一个人的陈述由多个句子组成，则必须分别处理这些句子才能进行意图识别。由于目前还没有可靠的方法来分割德语文本，因此在瑞士文本会议 SEPP-NLG[1]竞赛[TA21]中提交了一个定制的解决方案[Guh+21]。有关所开发方法的详细说明，请参见第 6.3 章。

文本分类的主要目的是将文本归入预定义的类别。在本研究中，文本分类主要用于确定情感，即语句的情绪。所开发的模型可按正面、负面或中性类别评估语句情感。情感分类在虚拟用户界面中的应用是多方面的。首先，它能让搜索服务提供商用户反馈立即做出反应。例如，如果一个人对语音识别系统的问候做出了负面反应，那么就可以建议他保持距离。另一个应用是自动评估对话。使用 HifeenesSentimentModels 可以对录制的幻灯片进行分析和评估。这种评估反过来又可以作为定性分析的基础，例如评估正面和负面陈述的数量。此外，对对话进行定性分析，特别是对带有负面情绪的发言进行定性分析，可以为可能出现的故障或发展潜力提供有价值的指示。在未来的实施中，还可以采用强化学习的方法，让虚拟用户界面语句的情绪作为学习信号，生成更有效的回应。撰写本论文，德语情感分类模型领域还存在很大的研究空白。因此，本论文专门为对话系统开发了这样一个模型。有关这一具体方法的详细信息，请参见第 6.4 章。

6.2 意图识别和命名实体识别

要通过对话管理（DM）实现用户与系统之间的有效互动，准确识别用户的意图至关重要。一旦识别出用户的意图，第二步就可以从用户的陈述中提取特定信息。这一过程对于触发有针对性的操作或详细处理用户问题特别有用。Liu 等人[Liu+19a]对包括意图识别和 NER 的各种软件包进行了评估。在所分析的四个模型中，有三个是依赖于现有互联网连接的在线服务，因此不在本研究的考虑范围之内。不过，开放源码框架 Rasa [Boc+J 7] 被证明是合适的，因此进行了评估，并与其他可用方法进行了比较。

值得一提的是，文献中已有解决方案可以完成这两项任务。这大大简化了开发和实施过程。

SEPP-N LG 竞赛网站 bs https://sites.google.com/view/sentenze-segmentation

文物检测

将用户的陈述分配给特定意图的分类模型适用分配意图。

有大量的分类模型可供选择。FastText jou+J 6; Boj+J 7]最初用于该虚拟用户界面，并在效率方面进行了优化。Rasa 框架为这项任务使用了一个专门为对话系统设计的特定模型 [Bun+20]。

这种方法的一个主要缺点是必须为每个类别提供足够的数据集。所需数据集的确切数量因记录的复杂性、与其他类别的相似性和具体实例而异。

在实际应用中，这种方法会导致两个核心问题：首先，必须有足够训练数据；其次，每次添加新意图或扩展现有意图的数据集时，都必须重新训练分类器。这会拖慢系统的更新和扩展进程。

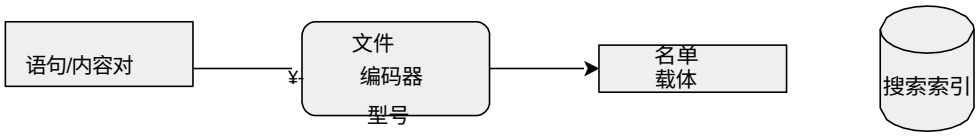


图 6.J : 创建搜索索引：为所有语句/意图对创建一个语句索引。
并存储在索引中。

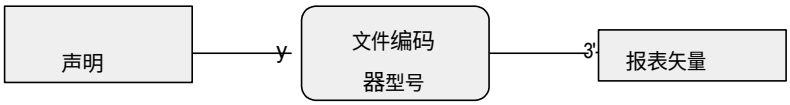
为了避免上述局限性和挑战，这项工作选择了另一种新式搜索方法。

图 6.1 和 6.2 说明了神经搜索的工作原理。第一步，如图 6 所示，创建搜索索引。作为该步骤的一部分，使用句子编码器（SentenceBert [RGJ 9] 和 [RG20]）为每个意图的示例语句生成所谓的句子嵌入。这种句子嵌入以向量的形式表示语句的含义。

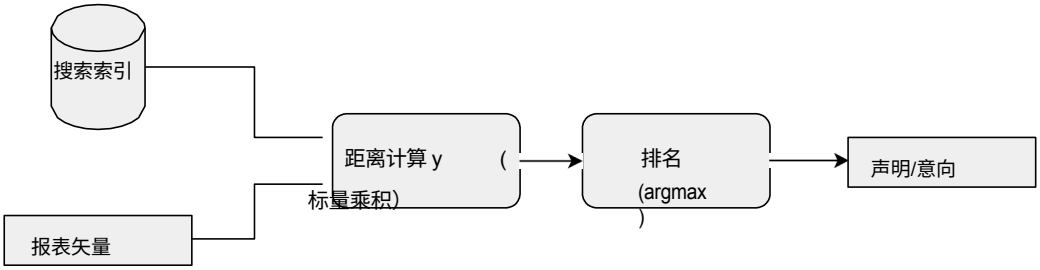
以这种方式生成的语句，其向量和相关的 Q IntentS 被分配给这些语句、会保存在创建的搜索索引中。这样，以后就可以将新的 BQnut 语句分配给相应的意图。

神经搜索方法的一个核心特征是测量两个语句之间的相似性。其方法计算各自语句向量之间的标量乘积或余弦相似度。图 6.2 展示了索引的搜索过程。首先，用户语句被句子编码器转换成一个向量。然后计算生成的搜索向量与搜索索引中所有存储语句的相似度。最后，从搜索索引中选出相似度最高的语句。

这种程序有以下优点



(a) 冰　stenSchüttwYdtheNu zers 语句被文档编码器模型编码成一个向量。



(b) 第二步，将语句向量与搜索索引中的所有语句进行比较。排序会返回与用户语句向量最相似语句和意图。

图 6.2：该图显示了神经搜索两阶段 PFO 流程。该过程从索引中输出与 NutZQF 语句最相似语句意图。

实时扩展：可以通过附加语句在 ZUF 运行时扩展搜索索引。

无需训练：与传统分类模型相比，无需对模型进行传统意义上的训练。

灵活性：即使只有一个示例的课堂也可以实现。如果系统需要回答一个非常具体的问题或陈述，这一点尤其有用。

不过，这种方法也有缺点：随着数据库中文本数量的增加，搜索的计算量也会线性增加。不过，使用 KD 树可以部分缓解计算量增加，从而提高在大型数据库中的搜索效率。

命名实体识别

在计算语言学中，NER 是指将文本元素转换为

Kategorien wie Personen, Organisationen, Orte, Datumsangaben automatisch identifiziert 并进行分类。在本 VUI 中，除其他事项外，还使用 NER 从一个人的陈述中提取其姓名。

本研究对 Flair 软件库 [Akb+J 9] 的两个模型进行了评估。Flair 库为德语两个特定的 NER 模型："ner-german-large" [SA2J] 和 "ner-german-base" [ABVJ 8]。这两种模型都能不同类别的实体：

- PER：人员姓名
- LOC：地点名称
- ORG: 组织名称

  杂项：活动、产品或艺术品名称

这两个模型在所使用的架构、模型大小和分类质量方面都存在明显差异。

表 6.J：两种德语 NER 模型的比较。由于执行时间明显更快，因此在 VUI 中使用了较大的模型。

模型		ner-german-base	ner-german-large
F/ 分数 内存消耗		87,94 .47 GB	92,31 2.24 GB
出口时间（平均值	00 次迭代）	26 毫秒	7 毫秒

使用 CoNLL-03 数据集[TD03]对模型进行了比较。CoNLL-03 是一个独立于语言的标准数据集，用于模型的训练和评估。在该数据集上，"ner-german-large "模型的 F1 得分为 92.3d，内存需求为 2.24GB；相比之下，"ner-german-base "模型在同一数据集上的 r1 得分为 87.94，内存需求为 0.47GB。

有趣的是，大型模型的处理速度更高。处理一个句子时，"ner-german-large "平均需要 7 毫秒，而 "ner-german-base "每句需要 26 毫秒（100 次运行的平均值）。

鉴于虚拟用户界面的要求，处理时间至关重要，因此 "ner-german-large "模式受到青睐。除了速度更快之外，该模型的分类质量也更高。只有当技能主动触发处理时，才会执行 NER。这一程序旨在节省计算能力，因为只有某些技能这一信息。

6.3 报表分类

本章是作者发表在[Guh+2J]上的一篇论文的译文摘要。

在开发 VUI 的同时，SEPP-NLG 共享任务 [TA2J] 作为一项竞赛在 202 年的 Swisstext 会议上公布。该竞赛的目的是开发一种模型，以便对句子进行分段，并在句子中加入"....."。

标点符号。这些模型可用于文本分析或在转录文本中插入标点符号。句子分割和标点符号预测是一项相对较新的任务，在本次竞赛之前，还没有系统能够可靠地这项任务。SEPP-NLG 共享任务包括两个子任务：

子任务 1 - 句子分割：针对完全无标点符号的小写文档。

子任务 2--标点符号预测：为完全无标点符号的小写文档预测所有标点符号，可能的标点符号p - (:-,?.0} ，其中 0 表示无标点符号。

这项任务是在 Europarl 语料库[Koe05]的德语、英语、法语和意大利语部分完成的，因为该语料库提供了多种语言的口语文本誊本。作者为这两项任务开发了模型。开发过程中使用了 Wolf 等人的转换器库[Wol+19]。这些模型和代码可公开获取[2]。

以往关于句子分割和标点预测的研究这一技术的不同应用领域。该任务通常被模拟为标记预测。，在神经网络和大规模神经语言模型的帮助下该技术的性能不断提高。

AAD14] 的工作一种更为传统的拼写和标点更正方法，在本例中是针对阿拉伯语的。作者报告说，在他们的数据集中，标点符号错误所有错误的 40%。这项任务的模型是一个标记分类，上下文窗口在 4 到 8 个单词之间。使用支持向量机和条件随机场 (CRF) 分类器，利用语音部分 POS) 和形态学信息进行分类。作者使用 CRF 分类器和 5 个词组的窗口大小取得了最佳结果，F/分数为 0.56。

[Che+16]试验了三种不同的神经网络架构，并使用预训练的 GIoVe [PSM14] 嵌入作为输入。在预测逗号、句号和问号的位置时，他们4 类分类中取得了最佳结果，Fi 得分为 0.54。

Treviso、Shulby 和 Aluisio [TSA17]研究的是句子分段，而不是句子之间的分段。研究人员还分析了在对患者进行语言障碍痴呆症症状检查时制作的叙述记录。他们使用了三个不同的葡萄牙语数据集。输入数据使用 POS 特征、单词嵌入和前音信息建模。然后，他们结合卷积和递归神经网络层，在两个评估数据集上取得了 0.7 和 0.8 之间的 F1 分数。

Schweter 和 Ahmed [SA19]也使用 Europarl 语料库进行了实验，但他们的实验结果并不令人满意。

他们的任务与本文所介绍的任务不同，即他们模拟的是

(2)https://github.com/oliverguhr/fullstop-deep-punctuation-prediction

3TED 是每年在加利福尼亚州举行的一次会议。演讲都是录制的，可在互联网上免费观看。自 2009 年以来，这些视频已配有各种语言的字幕。

仅通过预测输入文本中的每个点句子分割标记还是其他语言单位的一部分（例如表示缩写）来进行句子分割。预测结果由基于字符的模型生成，这些模型不仅接收要区分的标记，还接收本地上下文。他们使用多种语言（包括经常被忽视的语言，如波斯尼亚语、希腊语或罗马尼亚语进行工作，得分在 0.98 和 0.99 之间，其中 BiLSTM 模型的平均表现最佳。

Sunkara 等人[Sun+20]也在临床领域医疗 ASR 系统的输出进行研究。他们首先预测标点符号序列，然后预测每个输入单词的大小写，从而对标点符号和大小写进行联合建模。作者使用了一个预先训练好的转换器模型[Dev+J 9b; Liu+J 9b]。他们在医学数据上对模型进行了训练，并训练实际模型执行了一个任务适应步骤，即随机屏蔽文本中的标点符号。在点和逗号数据的预测中，Bio-BERT[Lee+19]作者对逗号的预测得分为 0.8J，对点的预测得分为 0.92）。

6.3.1 数据集

任务预测无标点小写文本中的句末（也缩写为 EOS）和标点符号。SeppNLG 共享任务组织者470 MB 的英语、德语、法语和意大利语文本。

该数据集由训练数据集和开发数据集组成。为模型评估提供了一个包含域内文本的测试数据集和一个包含域外文本的 "惊喜数据集"。

图 6.3 显示了子任务 2 中所有语言的标点符号标签分布情况。从图中可以，即使每个数据集中大多数死者的标点符号都是 "0"（为提高可读性，图 6.3 中省略了 "0"）这一事实，标点符号的分布也并不均衡。所有语言都遵循相同的分布模式，但也有细微差别。例如，逗号和圆点的频率差异在德语中尤为明显。德语中逗号的比例通常较高，这表明句子结构复杂。在其他语言对中连字符和冒号的分布也略有不同。

早期版本的子任务 2 还要求对标点符号"！"和"；"进行预测。在训练阶段，任务组织者将这些符号分配到点上，以考虑高度偏斜的分布。除上述标点符号外还包含其他标点符号的句子，例如括号

- 被任务组织者删除，原因是并非所有括号都格式正确（即并非每个开头括号都有一个结尾括号）。

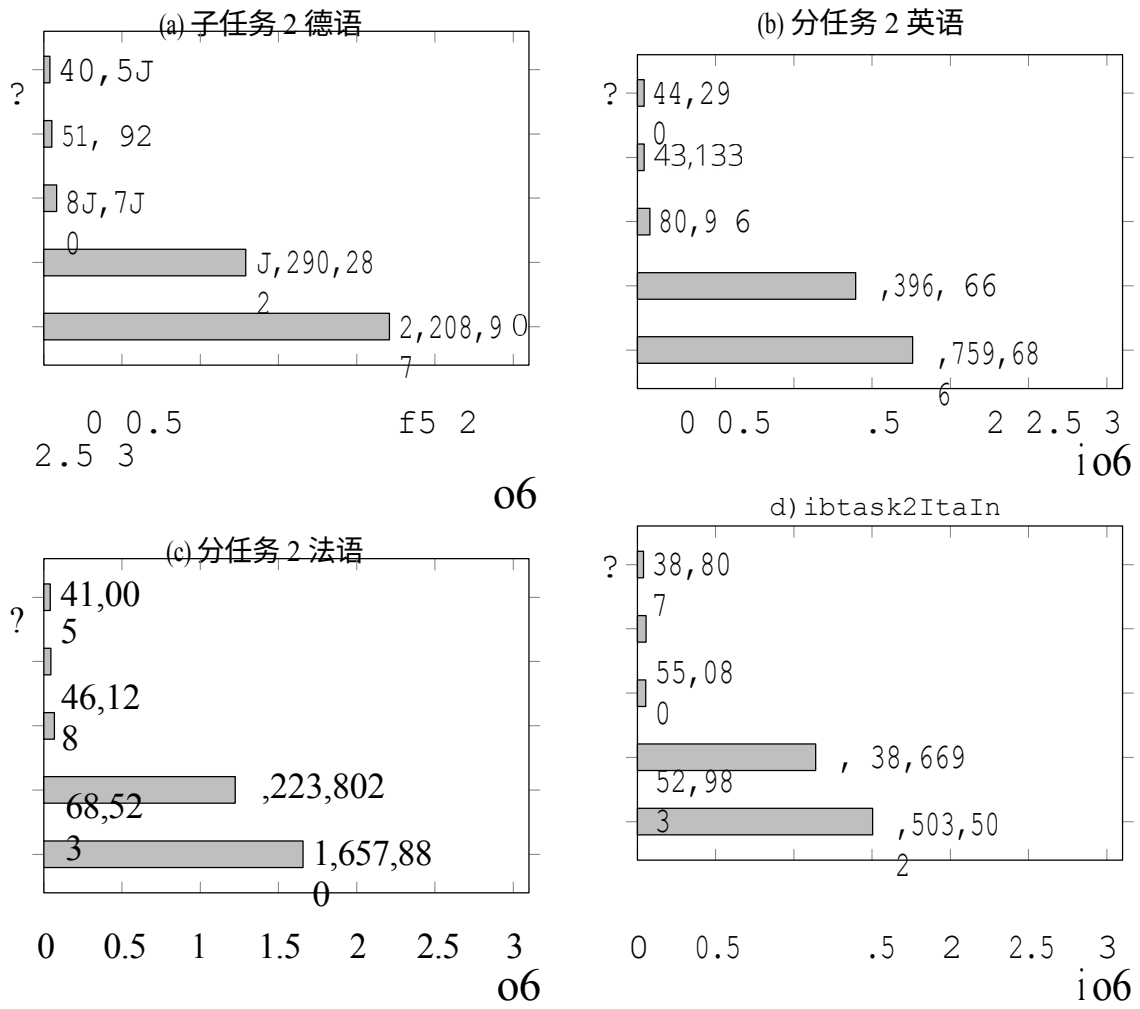


图 6.3：任务 2 训练集中四种语言的标点符号 Z6iChen 分布情况。 文档平均长度介于 0.378（意大利语）和 12.275 个字（法语）之间。

6.3.2 机型

转换器架构[Vas+J 7]和基于转换器语言模型的迁移学习[Dev+19b]在许多 NLP 任务中取得了显著的性能提升。因此，我们将重点放在了基于转换器的架构上，并对当前的一些语言模型和多语言迁移学习进行了研究。根据以前的工作，任务被模拟为标记预测。

不过，为了评估基于转换器的语言模型所带来的性能提升，我们还训练了第一个非神经基线模型（用于德语句子分割）：基于词袋、语音部分（POS）和本地上下文（+/- 2 个词组）特征的条件随机场模型。该模型的表现似乎明显优于为[第 5] 部分计算spaCy* 基线。但是，由于它性能被基于转换器的模型远远超过，因

此没有继续这种方法。

作为第二条基线，我们训练了一个简单的多语言贝尔特模型，并对其进行了技术分析。

---

https://spacy.io/。

[S]https://sites.google.com/view/sentence-segmentation/。

表 6.2：表中的所有基础模型都是在 1 0% 的特定语言数据上训练的，多语言模型则是在所有语言的 10% 数据上训练的。所有模型都使用 Adafactor 和 $^{4e-5}$ 的学习率了 3 个历时。对于任务 1，给出了 EOS 类别的 Fi 分数。对于任务 2，给出的是所有类别的宏观平均值+1。

基本型号	任务 1 F-1	任务 2 F
英语		
基于疏松的	0.849048	0.581294
伯特基不分区	0.885560	0.647669
bert-large-uncased	0.903943	0.699679
罗伯塔基地	0.921170	0.719705
XLM-ROBERTA-LARGE	0.932057	0.740402
岁伯塔-大	0.935672	0.742778
德国		
伯特基多语种无基化	0.931668	0.708220
dbmdz/bert-base-german-uncased	0.943437	0.746249
deepset/gbert-base	0.943571	0.753979
german-nlp-group/electra-base-german-uncased	0.950070	0.759387
法语		
伯特基多语种尤基化	0.881648	0.658968
卡门贝底	0.914799	0.702187
卡门贝/卡门贝大号	0.935436	0.756594
意大利语		
dbmdz/electra-base-italian-xxl-cased-generator	0.866070	0.496291
伯特基多语种无基化	0.867798	0.586234
dbmdz/bert-base-italian-cased	0.897765	0.658520
dbmdz/bert-base-italian-xxl-uncased	0.910585	0.693615
多语种		
伯特基多语种无基化	0.887909	0.683688
xlm-roberta-base	0.915930	0.716822
XLM-ROBERTA-LARGE	0.935946	0.753770

使用不同的技术对结果进行了分析，以改进这一基线。为此采用了三种不同的方案：数据增强、超参数优化和选择不同的架构或预训练模型。此外，还测试了各种预处理步骤，以去除特殊字符和 HTML 人工痕迹。不过，这对模型的结果没有影响。

作为模型选择的第一步，在每个任务的 10% 训练数据上一些单语和多语模型。随后，选出每种语言的最佳模型和最佳多语言模型，并在整个训练数据集上进行训练。这种方法有助于快速迭代避免了选择模型所需的漫长训练时间（在单个 GPU 上长达 20 小时）。我们选择了以下架构进行测试：

- 伯特 [Dev+19b]
- Distillbert [San+19]
- 伊莱克特拉 [Cla+20]

- 罗伯塔 [Liu+19b]
- XLM-Roberta [Con+20]
- 卡门柏乳酪 [马+20]

数据增强和超参数优化的初步实验表明，这些技术对模型的性能影响不大。所

$4e^{-5}$ sowie einer Batch-Größe von acht trainiert. Weiterhin wurden die Modelle mit einer
有 0% 和 00% 模型都使用 Adafactor [SSH 8a]训练3 个历元，学习率为

数值分辨率为 6 位，以提高训练速度。此外还进行了超参数优化。有关这方面的更多信息，请参见第 6.3.4 节中的烧蚀研究。

比较表 1 和表 2 的结果可以发现，当使用 100% 而不是 0% 的数据训练时，任务 0、% 和 % 的模型都有所改进。任务 2 的模型则提高了 3% 到 5%。

窗口方式

所有被选中的架构在可处理的标记数量方面都受到限制。通常，最大文本长度为 5J 2 个字节。由于大多数文档的长度都超过了这一限制（见图 6.3），因此一种策略来处理较长的序列。最简单的方法是在处理前将文本分成 200 个字的部分。200 个单词的数量是根据经验选择的，以考虑到单词被分解成多个标记的情况。这种方法的缺点是效率不高，因为大多数序列无法充分利用模型的 512 个标记。

表 6.3：连续序列之间有 00 个重复标记
提高了模型的性能。

重叠令牌	Fi -Score 任务 1
0	0.87893
J0	0.87933
J00	0.88556
200	0.88375

因此，每份文档首先要进行标记化处理，然后分割成 5J 2 个标记的序列。然而，这种方法和第一种方法一样，会产生以句子第一个单词开始或以句子第一个单词结束的序列，从而使模型无法进行预测。为了解决这个问题，我们使用了滑动窗口方法，并用不同的步长（类似于卷积神经网络的步长参数）进行了实验。这种

方法确保了模型的预测具有额外的背景。在训练过程中，使用英文伯特基模式 II 对 0% 的数据进行了网格搜索，以找到重叠窗口的最佳长度。根据表 6.3 中的结果，选择重叠窗口大小为 00 来训练模型。训练过程中的损失

对整个序列（包括重叠部分）进行计算。由于这种方法也能生成新的训练序列，因此也可用作数据扩增方法。

6.3.3 培训

表 6.4：子任务和任务 2 的所有模型都是在每种语言的完整数据集上训练的。表中给出了任务中句末类的-分数和任务 2 中类的平均 F/ -分数。

模型	测试语言	F/ 分数 任务 1	F/ 分数 任务 2
罗伯塔-大	EN	0.941992	0.772326
XLM-ROBERTA-LARGE	EN	0.938764	0.765496
电-基-德-基	EN	0.953894	0.795759
带有数据扩充功能的德语电子数据库	EN	0.954782	
大号卡门柏	FR	0.937222	0.778617
bert-base-italian-xxl-uncased	信息技术	0.919729	0.732624
	EN	0.945746	0.774601
xlm-roberta-large	EN	0.958591	0.813861
	FR	0.941974	0.781834
	信息技术	0.934144	0.761775

表 6.2 显示了比较 0% 模型的训练结果。所有在任务 1 中表现最佳的模型在任务 2 中也表现最佳。我们为英语选择了 XLM-RoBERTa Large 和 RoBERTa Large 这两个模型，因为它们都取得了相似的结果。在德语方面，基于 Electra 的模型取得了最佳结果。法文选择了 Camembert Large，这是一个基于 RoBERTa 的 3.35 亿个参数的模型，其性能明显优于拥有 1 千万个参数的 Camembert Base。巴伐利亚州立图书馆数字图书馆团队（dbmdz）发布了两种不同的基于 Bert 的意大利语模型，其中 XXL 版本的模型在更大的语料库中进行了训练，取得了最佳结果。多语种 XLM-RoBERTa 基础模型 QF 在参数数量相同的情况下比旧版多语种 Bert 模型取得了更好的结果。该模型的较大版本拥有 3.35 亿个参数，取得了最好的多语言结果，特定语言模型相当。在多语言数据集上对多语言模型的结果进行了分析。

所选模型在每个任务的完整训练数据集上进行了训练，并使用开发数据集进行了评估。在这两个任务中，大型多语言 XLM-RoBERTa 的表现优于所有特定语言模型。因此，基于 XLM-RoBERTa 的模型被提交给任务 1 和任务 2。对于 ita

在语言方面，基于 XLV-RoBERTa 的模型取得了明显优于最佳特定语言模型的结果。然而，

对于其他语言来说，性能提升。基于德语 Electra 的模型与 XLM-RoBERTa 的结果相当，尽管它使用了 0 万个参数，而 XLM-RoBERTa Large 使用了 5.5 亿个参数。这，通过使用较小的参数，仍有可能提高性能。

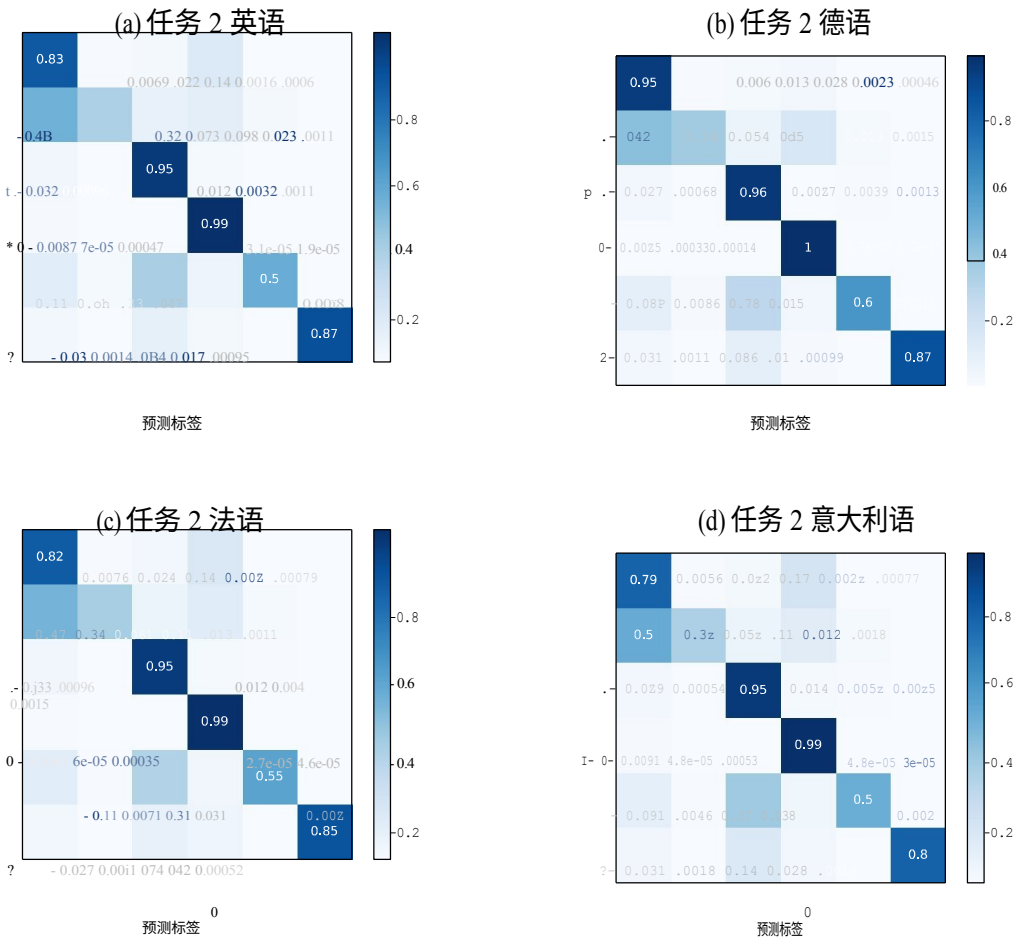


图 6.4：混淆矩阵　　的　的混淆矩阵　　XLM 基于　　RoBERTa　　所有数值均已四舍五入。

单语模式。

由于多语言模型的性能几乎优于所有单语言模型，因此这些模型被用于子任务 1 和 2。此外，还提交了一个较小的德语模型，以评估其在测试数据集和超采样数据集上的性能。

FullStop 多语言任务 1 该模型基于 5.5 亿个参数的 XLM-RoBERTa Large 模型，并在任务 1 的标注数据上进行了训练。在所有四种语言中，该模型在测试集中的平均 F 值为 0.94，在突击测试集中的平均 Fi 值为 0.78。

FullStop 德语任务 2 该模型基于 1.1 亿个参数的德国 Electra Base 模型。该模型在任务 1 的标注数据和一个额外的数据集上进行了训练，该数据集包括德国联邦

议院（Bundestag，134 MB[6]）的发言数据和莱比锡语料库（Leipzig Corpora Collection，245 MB[7]）的文本抓取数据，其中包含新闻文本和维基百科文章的混合数据。

6https://github.com/Datenschule/offenesparlament-data
(7)https://wortschatz.uni-leipzig.de/de/download/German

就德语而言，该模型在测试数据集中的 F1 值0.95，在突击测试数据集中的 F/ 值达到 0.80。

表 6.5：FullStop 多语种任务 2 模型的每类 F/ 结果
dev 数据集。

标签	EN	EN	FR	IT
	0.8?9	0.945	0.83?	0.798
	0.42s	0.43b	0.43?	0.42?
	0.948	0.96\	0.945	0.942
0	0.993	0.997	0.992	0.989
	0.575	0.652	0.620	0.588
?	0.890	0.893	0.87?	0.832
YäCLO 1	0.775	0.8\4	0.782	0.762

FullStop 多语言任务 2 该模型也是XLM-RoBERTa Lar- ge，并针对任务 2 进行了训练。如图 6.4 和表 6.5 所示，该模型在所有四种语言的句子分割方面都表现良好。相比之下，冒号和连字符的性能较低。可以认为由于数据集的特点造成的，如第 6.3 节所述。

6.3.J，因为在所有语言的训练数据中，连字符和冒号都很少见。直观地说，这并不奇怪，因为连字符和冒号在很多情况下是可选的，即可以用逗号或句号代替。它们的使用规则不仅与语法和句法有关，而且与文体有关。通过使用相反示例进行有针对性的训练，可以提高模型的性能。该模型在测试数据集中的平均 r1 值为 0.78。与其他模型

在域外数据的情况下，使用域外数据的惊喜数据集的性能下降到平均值 0.6。

使用 Nvidia 3090 GPU 对整个测试和惊喜集（470 MB）进行推理，每个多语种 FullStop 模型大约需要一个小时。

用于预训练的数据类型和数量对最终模型的性能有重大影响。

表 6.2 显示，在意大利语任务 2 中两个基于 Bert 的单语模型之间的差异为 5%。这两个模型使用相同的 1.1 亿个 Bert 架构参数，但训练时使用的语料库大小不同。bert-base-italian-uncased" 模型使用 3GB 的语料库进行训练，而 bert-base-italian-xxl-

uncased" 模型使用 81GB 的语料库进行训练。对于其他转换器架构例如 xml-roberta 和 clark2020electra，更大的语料规模对模型性能的积极影响也得到了证实。

这两项任务都受益于多语言模型和训练数据。

令人惊讶的是，基于 XLM Roberta 的多语种模型的表现优于所有单语种模型，尽管之前的多语种 Bert 模型在大多数情况下表现都优于单语种模型。

语言特定的情况下，XLM-Roberta Lar-ge 的表现优于它们。假设的解释是，XLM-Roberta Lar- ge 单独使用的参数较多。为了验证这一假设，我们训练了一个基于 XLM-RoBERTa 的单语英语模型和另一个基于 RoBERTa 的单语英语模型。如表 6.4 所示，这两个单语模型的表现都优于经过多语言训练的 XLM-RoBERTa 模型。虽然直接解释多语言模型的优越性能，但需要强调的是，这与之前的工作[Mul+2]是一致的，该工作证实（对于 mBERT）多语言模型的下层通过代表不同语言的语言知识来充当多语言编码器。如果在这里也是如此，那么更多的多语言训练示例确实可以标点符号的表现。第 6.3.4 节中的消融研究结果也指向这个方向。不过，这些假设还需要经验验证。

标点符号模式针对特定领域，稳健的标点符号预测在多个数据集上进行训练。

进行训练的数据集（Europarl）由来自单一领域（即政治演讲）的数据组成，在惊喜集上获得的结果降低了模型在其他领域文本上的性能。 为该任务开发的模型的性能从域内测试集的 0.94（所有语言的平均值）下降到域外惊喜集的 0.78。参加联合任务的其他模型也出现了类似的性能下降。

6.3.4 消融研究

机器学习领域的消融研究是一种系统调查方法旨在了解单个模型或系统组件的重要性。通常，模型或特征的某些部分会被特别删除，以分析它们对系统性能的影响。这样做的目的是区分基本组件和不太重要或多余的组件。
通过移除系统部件，可以深入了解模型的功能。
erlangt werden. Dies ermöglicht die Optimierung und Vereinfachung der Modelle durch

消除无效部分，并有助于加深对互动的理解。

不同组成部分及其对总体结果的贡献。

在这项工作中，我们进行了实验，以测试影响模型结果的各种因素。本节将介绍这些实验的问题和结果。

每个模型的最佳超参数是什么？

使用 Optuna 框架[Aki+J 9]对 Adam 优化器了超参数搜索，在德国 Electra 基本

模型上运行了 200 次。为超参数搜索配置了以下搜索空间：

学习率介于 0-* 和 0- 之间，最多 5 个训练历元，批量大小介于 2' 和 27 之间，权重衰减*介于 0-1 和 0-' 之间，亚当ε介于 10-* 和 0-(1)* 之间。这些设置与学习率为 4e5 的 Adafactor [SSH 8b] 了比较。对于这两种优化，任务 1 和任务 2 的模型都是用 0% 训练数据进行训练的。比较结果6.6。Adafactor 达到了与 Adam 相同的性能，但省去了耗时的超参数搜索，因此决定所有模型都使用 Adafactor。

表 6.6：使用优化超参数的 Adam 与 Adafactor 的比较结果显示，两者的 F1 分数只有微小差异。

任务	Adafactor	亚当	p.p. 的差异
2	0.95007 0.75939	0.95087 0.75587	-0.0008 +0 00352

有可能用一个模型同时完成这两项任务吗?

任务 2 的标签是任务 1 的标签的超集，因此，为任务 2 训练的模型可用于1。通过将句末标签". "和"? "赋值为标签 1（句末）将所有其他标签赋值为 0（无句末），可以改变任务 2 的分类结果。表 6.7 中的结果表明，这种方法仅略微F1 分数。对于许多应用而言，只需训练一个能同时处理所有四种语言任务的模型。我们为这一联合任务训练并提交了两个不同的模型，作为任务的特定模型
结果略有改善。

表 6.7：比较了 "FullStop 多语言任务 "模型和 "FullStop 多语言任务 2 "模型的定制输出结果，以便对任务 ZU 的标签进行 eDtSpeak。这种方法使 Fp-SCOre 略有降低。

语言	任务模式	任务 2 的模型
EN	0.945746	0.94J686
EN	0.958591	0.955926
Ir	0.941974	0.938254
它	0.934144	0.93085J

您需要深度模型来完成这些任务吗?

共享任务的目的不是为了优化推理和训练效率。不过，我们测试了是否有必要Bert模型的所有两层。为此，对 0% 的英语数据建立了 3、6、9 层的模型。

---

[8]权重衰减（Weight Decay）是一种正则化技术，用于优化神经 NQtZ，以防止过度拟合。其目的是通过添加损失函数来保持模型菜肴 kIRiD 值。

和 0 层进行了任务训练。为了保持结果的可比性，使用了与本节所述所有其他模型相同的超参数。

6.3.2 所述。表 6.8 中的结果表明，采用这种简单的减少层的方法，可以去掉最后一层中的 /4，同时 99% 的模型性能（ri 分数）。建议在进一步研究中
更先进的优化技术，以提高模型的效率。
改善。

表 6.8：按不同步骤减少轮班次数伯特基础模型的 F/ 分数。这些数值适用于在 10%的数据基础上训练的英语模型。

层数	参数	Fi -Score 任务 1
3	45,J02,338	0.74758
6	66,365,954	0.84408
9	87,629,570	0.87776
2	\08,893,\ 86	0.88556

6.3.5 成果

该出版物[Guh+21]表明，基于转换器的架构可成功应用于标点符号和句末预测任务。令人惊讶的是，单模型的性能优于多语言模型，这表明这些模型可以在不同语言之间知识，即不同语言中存在共同的规律性，模型中对这些规律性的表示对其性能有有利影响：首先，应缩小模型的规模。两个 "FullStop 多语言 "模型都使用了 5.5 亿个参数，这导致了计算密集型推理。在序列中
首先，在第一次训练测试中提出了减少参数数量的初步方法。其次，应提高训练数据域外模型的性能。共享任务的惊喜数据集显示，来自未知领域的文本的性能下降。

简化研究人员和开发人员所开发模型的使用，我们开发了一个 Python 库，只需三行代码就能简单地集成到自己的项目中。该项目的资源库包含底层代码和训练有素的任务 2 模型。经过训练的模型也可在 "拥抱面孔 "平台上获取 (10) 。该模型在 2022 年 11 月至 2024 年 2 月期间被下载了约 520 万次。

此外，MOdelIvOnVandeghnste and Guhr [VG23]也进行了改编，以针对荷兰语进行优化。该定制模型也可在'拥抱脸庞'平台上使用。

这些发展不仅表明了这一特殊模型的相关性和对它的兴趣，也表明了对有效和易于整合的模型的需求日益增长。

---

https://gith.com/oIiverguhr/deepmultilingualpunctuation
'*https://huggingface.co/oIiverguhr/fullstop-punctuation-multilang-large
https://huggingface.co/oIiverguhr/fullstop-punctuation-multilingual-sonar-base

为不同语言的文字处理任务提供解决方案。

6.4 基于文本的情感分析

以下部分作者的研究成果，该成果英文版本为 [Guh+20]。本文对其内容进行了翻译和改编。

为这项工作开发的虚拟用户界面将用于各种护理场景。在这些场景中，虚拟用户界面必须能够适应用户的需求并对人的情绪做出反应。可以认为，从用户的语音输入中捕捉和理解情感信号能力不仅对服务机器人的接受度至关重要，而且对处理用户提供强烈情感反馈的情况也至关重要。

在语音界面中使用了两种情感分析应用程序

认为：

> 其目的是对用户发言的情绪进行分类。这样，虚拟用户界面的对话管理器就能对用户的正面和负面反馈做出反应，并在必要时调整可能的语句。
>
> 此外，用户的话语及其情感标签随后可用于评估对话和疏散用户的反应。这一概念很，[Che+18] 和 [Fan+18] 等 VUI 也将情感分析用于这些任务。

为情感分析开发可靠的机器学习模型需要标注训练数据。然而，现有的德国情感数据集 WiQ *POtTS* [Sid16]、*SBI0k* [Cie+J 7] 和 *GermEval-201* 7 [Woj+J 7]合起来只有 39,000 SÖtZQ。

相比之下，常用的英语数据集包含 WiQ /Vö8 [Maa+]。

50,000 和 VOIP 数据集 [ZZLJ 6] 598,000 标记元素。另一个重要方面是，*PotTS* 和 SB 0 都包含有注释的推文，而 *GermEval-201* 7 则包含推文和 Facebook 帖子的组合。因此，所有可用的德语数据集都反映了社交媒体语言，而不是普通德语。

为了克服这些限制，通过抓取德国网站 *holidaycheck.de* 上的酒店评论和网站 /i/f7/StOLtS.de 上的电影评论来收集额外的数据。这些评论中包含的评分经过调整，以与现有数据集的三种情感类别（负面、中性、正面）相匹配。同样的过程也应用于*分数*语料库[Sän+J 6]，该语料库由应用程序评分组成。此外，通过添加来自

Leipzi$ 语料库[GEQJ 2]的文本，对*中性*类别进行了扩展。在最后的写作中，数据集通过添加侮辱等高度情绪化词汇的语句进行了扩展。这些语句是在服务机器人现场实验中记录的[Pos+12；

随后，我们将所有数据集合并在一起，创建了一个大型、多样化的德语情感数据集，该数据集能更好地近似反映可引导服务机器人的语句类型。然后，对 FastText Lou+J 6] 和 BERG [Dev+19c] 两种不同类型的机器学习模型进行了训练和性能评估。数据集、训练模型和源代码均可公开获取 *。

德语有几种情感词典，例如 BAWL-R、SentiWS 和 GermanPolarityClues [Vö+09; RQHJ 0; Walt 0]。

Cieliebak 等人在其 *SBI* OR 数据集上训练了基于 SVM 和 CNN 的模型，并在其数据集上取得了 65.09 的最大 F 值（正负 F 值的平均值）。

德语情感分类的一项举措是基于方面的情感分类的 *GermEval-201* 7 共享任务 [Woj+J 7]。该数据集包含与德国铁路相关的语篇。该共享任务提供了多个子任务，如相关性过滤、基于文档和方面的情感分类以及意见目标提取。在文档层面的情感分类方面，表现最好的系统的 ri<.<-值为 74.9 [NgKJ 8]。

6.4.1 数据

表 6.9：该表显示了所使用的数据集列表和每个类别的数据点数量。这些数据指的是预处理后的数据集。

数据集	阳性样品	中性样品	阴性样本	样本总数
情感	188	28	1,090	1,306
电影发行	40,049	0	15,610	55,659
GermEval-20a 7	1,371	16,309	5,845	23,525
假日检查	3,135,449	0	388,744	3,524,193
莱比锡-维基百科	0	1,000,000	0	1,000,000
PotTS	3,448	2,487	1,569	7,504
履行机构 0k	1,716	4,628	1,130	7,474
惊吓	538,103	0	197,279	735,382
总和	3,720,324	1,023,452	61 1,267	5,355,043

该数据集三个类别：正面、中性和负面。新类别非常重要，因为并非所有用户的陈述都可能包含正面或负面情感。像 "你叫什么名字？"或 "你能帮我吗？"这样的问题不包含任何情感，应归类为中性。*PotTS*、*SBI OK* 和 *GermEval-2017* 这三个人工注释数据集也使用了这些类别。VOS [PLV02]方法被复制，以扩大领域的多样性并提高所使用数据集的总体数量。对于酒店、电影和应用程序的评级，用户可以在

文本中添加星级。 这些星级从零星到五星不等，零星或一星是最负面的，一星是最正面的。

[1]*https://github.com/oliverguhr/german-sentiment